1. Introducción

En esta actividad comenzamos con el desarrollo de un prototipo de una ambulancia, el cual posteriormente tendrá diferentes funciones para poder controlar su movimiento. La idea principal del proyecto es que las llantas del lado izquierdo puedan ser controladas de manera independiente a las del lado derecho, para así poder tener un mayor control sobre la dirección del carrito.

En esta primera etapa nos enfocamos principalmente en las llantas del lado izquierdo, utilizando un Arduino UNO, un controlador de motores L293D, botones y motores. Los botones funcionan como la forma en la que el usuario le indica al carrito qué movimiento realizar, mientras que el Arduino recibe estas señales y manda las instrucciones correspondientes al motor. En este avance se buscó que las llantas pudieran moverse hacia adelante y hacia atrás, dejando para una siguiente parte la implementación completa de los giros.

Con esto se empieza a formar la base del sistema de control de la ambulancia, ya que más adelante se podrán integrar los dos lados de las llantas y hacer que el carrito pueda cambiar de dirección dependiendo de las instrucciones que reciba.

2. Objetivo

El objetivo de esta actividad es desarrollar la primera parte del sistema de movimiento de un prototipo de ambulancia utilizando Arduino. Se busca controlar de manera independiente las llantas del lado izquierdo mediante botones, logrando que los motores puedan girar hacia adelante o hacia atrás dependiendo de la instrucción que se envíe.

También se busca comprender cómo utilizar el Arduino junto con el controlador L293D para controlar motores, además de aprender como las entradas de los botones pueden utilizarse para generar diferentes movimientos. Esta parte servirá como base para posteriormente integrar el control de las llantas del lado derecho y lograr que el carrito pueda moverse y cambiar de dirección de una manera más completa.

3. Materiales

Para realizar esta parte del proyecto se utilizaron los siguientes materiales:

- 1 Arduino UNO
- 1 controlador de motores L293D
- 4 motores de corriente directa
- 4 botones pulsadores
- 4 resistencias
- 1 protoboard
- Cables jumper
- Fuente de alimentación
- Computadora para programar el Arduino
- Cable USB para conectar el Arduino a la computadora

4. Metodología

Al iniciar el sistema, el Arduino está programado para revisar de manera constante el estado de los cuatro botones mediante la lectura digital. El usuario interactúa presionando los botones para indicarle al prototipo la acción que debe realizar. Al presionar el botón para avanzar o retroceder, el Arduino envía la señal correspondiente al controlador L293D, el cual establece la dirección y acciona los motores. Si no se detecta ningún botón presionado, se ejecuta la instrucción para apagar los motores.

4.1 Conexión de los botones

Se utilizaron cuatro botones pulsadores y cuatro resistencias, los cuales fueron colocados en la protoboard. Las conexiones hacia el Arduino se distribuyeron en los pines digitales para poder recibir la señal correspondiente a cada movimiento:

- Botón Izquierda: Conectado como entrada al pin 2.
- Botón Atrás: Conectado como entrada al pin 3.
- Botón Adelante: Conectado como entrada al pin 4.
- Botón Derecha: Conectado como entrada al pin 5.

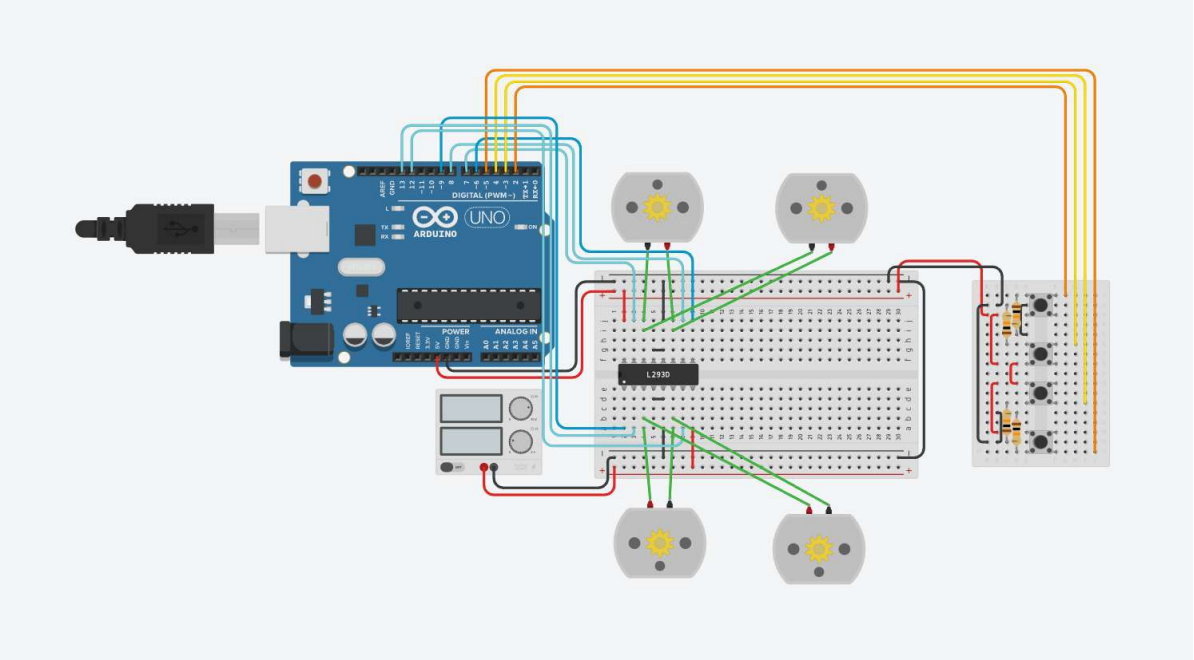
4.2 Funcionamiento y conexión de los motores y el controlador L293D

Los motores de corriente directa (DC) funcionan girando en un sentido u otro dependiendo de la polaridad de la energía que reciben. Para poder controlar este giro desde el Arduino sin tener que cambiar los cables físicamente, se implementó un controlador de motores L293D. Este circuito integrado funciona internamente como un puente H, lo que permite cambiar la dirección en la que fluye la corriente y, por lo tanto, decidir si el motor gira hacia adelante o hacia atrás. Además, el L293D es necesario porque aísla la etapa de control (el Arduino) de la etapa de potencia; los motores requieren más corriente de la que el Arduino puede entregar, por lo que el controlador toma la energía de una fuente de alimentación externa y la dirige a los motores de forma segura.

Para lograr controlar las llantas izquierdas, los pines del controlador L293D se conectaron de la siguiente manera:

- **Pin Enable (Habilitador 1,2):** Se conectó al puerto digital 6 del Arduino (definido en el código como motorEna). Este pin sirve para encender o apagar el canal del motor y nos permite controlar la velocidad utilizando modulación por ancho de pulso (PWM).
- **Pines Input 1 e Input 2 (Entradas de control):** Se conectaron a los puertos 7 (motorIn1) y 8 (motorIn2) del Arduino, respectivamente. Estos pines reciben las instrucciones lógicas. Como se observa más adelante en el código, si motorIn1 está en estado HIGH y motorIn2 en LOW, el motor gira hacia adelante. Si se invierte y motorIn1 es LOW mientras motorIn2 es HIGH, el giro es hacia atrás (dirección opuesta).
- **Pines Output 1 y Output 2 (Salidas al motor):** Estos pines se conectaron directamente a las terminales de los motores de corriente directa del lado izquierdo. Son los encargados de transmitir el voltaje con la polaridad correcta dictada por las entradas.
- **Pines GND (Tierra):** Se conectaron a la línea negativa de la protoboard para unificar las tierras del Arduino, la fuente de alimentación externa y el controlador L293D, cerrando así el circuito.
- **Pines de voltaje (Vcc1 y Vcc2):** Uno de estos pines se conecta a los 5V del Arduino para la alimentación lógica del chip, y el otro se conecta a la línea positiva de la fuente de alimentación externa, la cual es la encargada de suministrar la energía con mayor potencia directamente a los motores.

4.3 Diagrama de conexión



5. Código utilizado

```ino
int botonIzquierda = 2;
int botonAtras = 3;
int botonAdelante = 4;
int botonDerecha = 5;

int motorEna = 6; // Control de velocidad (PWM)
int motorIn1 = 7; // Control de dirección 1
int motorIn2 = 8; // Control de dirección 2

// Velocidad base del motor (0 a 255)
int velocidad = 255;

void setup() {
  pinMode(botonIzquierda, INPUT);
  pinMode(botonAtras, INPUT);
  pinMode(botonAdelante, INPUT);
  pinMode(botonDerecha, INPUT);

  // Configurar los pines del L293D como salidas
  pinMode(motorEna, OUTPUT);
  pinMode(motorIn1, OUTPUT);
  pinMode(motorIn2, OUTPUT);

  // Iniciar con el motor apagado
  digitalWrite(motorIn1, LOW);
  digitalWrite(motorIn2, LOW);
  analogWrite(motorEna, 0);
}

void loop() {
  // 1. Leer el estado de los 4 botones
  bool estadoIzquierda = digitalRead(botonIzquierda);
  bool estadoAtras     = digitalRead(botonAtras);
  bool estadoAdelante  = digitalRead(botonAdelante);
  bool estadoDerecha   = digitalRead(botonDerecha);

  // 2. Establecer la velocidad en el pin Enable
  analogWrite(motorEna, velocidad);

  // 3. Evaluar qué botón está presionado y mover el motor
  if (estadoAdelante == HIGH) {
    // Girar hacia adelante
    digitalWrite(motorIn1, HIGH);
    digitalWrite(motorIn2, LOW);
  }
  else if (estadoAtras == HIGH) {
    // Girar hacia atrás (dirección opuesta)
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, HIGH);
  }
  else if (estadoIzquierda == HIGH || estadoDerecha == HIGH) {
    // Al tener un solo motor, estos botones lo detienen por ahora.
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, LOW);
  }
  else {
    // Si no hay ningún botón presionado, apagar el motor
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, LOW);
  }

  // Pequeña pausa para estabilizar la lectura de los botones (debounce básico)
  delay(20);
}
```

6. Explicación del código

El código en Arduino comúnmente se divide en dos secciones, “setup” y “loop”. En la sección setup, se ponen las instrucciones que se desean ejecutar únicamente una vez cuando el sistema inicie, y en loop, el que se desea repetir mientras el Arduino esté encendido.

Lo primero que se hizo fue crear cuatro variables, para que a estas les fueran asignadas un puerto del Arduino para cada una (2,3,4 y 5), estas variables se nombraron: “botonIzquierda” (movimiento a la izquierda), “botonAtras” (movimiento hacia atrás), “botonAdelante” (movimiento hacia adelante) y “botonDerecha” (movimiento hacia la derecha).

También crearon 3 variables las cuales cumplen con la función de controlar el motor mediante un L293D, el cual es un controlador de motores. Estas variables fueron “motorEna”, la cual está conectada al puerto 6 del Arduino y sirve para controlar la velocidad utilizando modulación por ancho de pulso (PWM), y “motorIn1” y “motorIn2”, las cuales están conectadas en el puerto 7 y 8 del Arduino y deciden hacia que sentido gira el motor.

La última variable creada fue “velocidad”, a la cual se le asignó el valor 255, la cual es la velocidad máxima.

Dentro de la sección setup, primeramente utilizamos la función pinMode para hacer que los cuatro botones sean entradas utilizando INPUT, así cuando el botón sea presionado, el Arduino podrá detectar la señal.

También, algo similar se hizo con “motorEna”, “motorIn1” y “motorIn2”, solamente que estas fueron configuradas para ser salidas utilizando OUTPUT, es decir, estas variables recibirán señal del Arduino para controlar el motor usando el controlador.

Para finalizar, se configuró en “motorIn1” y “motorIn2” un estado LOW, y “motorEna” en 0, para que así cuando el programa sea iniciado el motor no se mueva a menos de que reciba una instrucción.

Para la sección loop, primeramente se colocó la función “digitalRead” con el fin de que se esté revisando constantemente que está pasando con los cuatro botones. Lo que se halle en cada revisión, será guardado en las nuevas variables: “estadoIzquierda”, “estadoAtras”, “estadoAdelante” y “estadoDerecha”.

Después, para configurar la velocidad del motor se uso la parte del código “analogWrite(motorEna, velocidad);”, en la cual el motor recibe la velocidad máxima de acuerdo a una serie de condiciones “if” y “else if”, con las que se determina el movimiento del motor.

Si el botón presionado es el de movimiento hacia adelante, se ejecuta:
“digitalWrite(motorIn1, HIGH);

“digitalWrite(motorIn2, LOW); “

Si el botón presionado es el de movimiento hacia atrás, se ejecuta lo contrario:

“digitalWrite(motorIn1, LOW);

digitalWrite(motorIn2, HIGH);”.

Debido a que en esta entrega se está trabajando únicamente con las las llantas de la izquierda, el código de los botones que serán utilizados para dar giros a la derecha e izquierda aún no está configurado para ello, y en caso de presionar cualquiera de estos dos, el motor se apagará debido a que se ejecuta:

“digitalWrite(motorIn1, LOW);

digitalWrite(motorIn2, LOW);”

Si ningún botón es presionado, igualmente lo anterior será ejecutado.

Para finalizar con esta sección, se usó “delay(20);”, una función que hará una pausa de 20 milisegundos que evitará que se detecten varias pulsaciones del botón dentro de una sola.

7. Resultados

Después de cargar el código y realizar conexiones, se realizaron distintas pruebas con el fin de comprobar si el prototipo funciona como se espera. Con ellas, primeramente pudo comprobarse que al presionar el botón de movimiento hacia adelante, efectivamente el motor giraba en un sentido, y que este cambia al presionar el botón de movimiento hacia atrás, con lo que pudo concluirse que el Arduino detecta los botones correctamente y que está enviando señales al controlador para que el motor actúe.

Botón presionado	Resultado
Ninguno	El motor no se mueve
Adelante	El motor gira hacia un sentido
Atrás	El motor gira en un sentido contrario
Izquierda	Aún no se implementa la función, no se mueve
Derecha	Aún no se implementa la función, no se mueve

8. Conclusión

En este avance logramos crear un sistema de control, en esta ocasión únicamente para las llantas izquierdas de un prototipo de ambulancia. Para la realización de esto, se usaron distintas herramientas como un Arduino, cuatro botones, motores y un controlador L293D.

Con el programa pudimos controlar el giro del motor usando los botones como medio, con lo que se pudo lograr movimiento hacia adelante y hacia atrás de las llantas de acuerdo a lo que el usuario eligiera. Con las pruebas pudimos comprobar que el sistema funcionaba de una manera correcta, respondiendo bien a las pulsaciones de los botones y a las instrucciones ejecutadas, con lo cual se pudo lograr el objetivo de esta actividad.

9. Referencias

SparkFun Electronics. (s.f.). *Pull-up resistors*. SparkFun Learn. Recuperado el 28 de septiembre de 2026, de https://learn.sparkfun.com/tutorials/pull-up-resistors/all

The Engineering Projects. (2017, julio). *Introduction to L293D*. Recuperado el 28 de septiembre de 2026, de https://www.theengineeringprojects.com/2017/07/introduction-to-l293d.html